

A CASE STUDY ON

KL ERP SYSTEM

An Integrated Academic and Administrative Management Platform

Roll no:2420090145-Pardheev

Software Engineering — Case Study

User Stories • Agile Process Model • Java Implementation • Evaluation Strategy

August 2026

1. Introduction

1.1 What is an ERP System?

Enterprise Resource Planning (ERP) refers to a category of software that integrates and manages an organization's core business processes on a single, unified platform. Rather than operating through disconnected departmental tools, an ERP system shares information across departments in real time, automates repetitive workflows, and improves overall organizational efficiency. The defining idea behind ERP is simple: one platform, many departments, and one shared, real-time source of data.

1.2 What is the KL ERP System?

The KL ERP System is a proposed integrated academic and administrative management platform designed for use in educational institutions. It is intended to bring together the many disconnected processes that a college or university runs day to day — admissions, course registration, attendance tracking, timetable management, examinations, fee payment, and results and communication — into a single, coherent system accessible to students, faculty, and administrative staff alike.

Today, many of these processes are still handled through separate spreadsheets, paper registers, or unconnected tools, which leads to duplicated effort, delayed communication, and inconsistent records. The KL ERP System addresses this by centralizing academic and administrative data, giving every stakeholder role-appropriate access to the information they need, when they need it.

1.3 Purpose and Scope of this Case Study

This case study documents the design and engineering rationale behind the KL ERP System. It walks through the user needs the system is built to satisfy, the process model chosen to develop it, the technology stack selected for implementation, the strategy for building and rolling out the system, and the approach used to evaluate whether it meets its goals. Two UML diagrams — a use case diagram and a class diagram — are included to illustrate the system's functional scope and structural design.

2. User Stories

A user story is a short, simple description of a software feature written from the end user's perspective. It helps the development team understand real user needs before any design or code is written, and it follows a consistent format:

"As a [user], I want [functionality], so that [benefit]."

The KL ERP System is designed around three primary actors, each with distinct goals:

Student
Learners accessing academic records, courses, and payments.

Faculty

Instructors managing attendance, materials, and grading.

Administrator

Staff overseeing records, reports, and institutional data.

2.1 Student User Stories

#	Feature	User Story
1	Secure Login	As a student, I want to log in securely so that I can access my academic information.
2	Course Registration	As a student, I want to register for courses online so that I can select my subjects easily.
3	Attendance Tracking	As a student, I want to view my attendance so that I can monitor my academic progress.
4	Examination Results	As a student, I want to check examination results so that I can evaluate my performance.
5	Online Fee Payment	As a student, I want to pay fees online so that I can complete payments conveniently.

2.2 Faculty User Stories

#	Feature	User Story
1	Record Attendance	As a faculty member, I want to record attendance so that student attendance can be tracked accurately.
2	Upload Materials	As a faculty member, I want to upload course materials so that students can access learning resources.
3	Enter Marks	As a faculty member, I want to enter marks so that students can view their academic performance.

2.3 Administrator User Stories

#	Feature	User Story
1	Manage Records	As an administrator, I want to manage student records so that all student information remains updated.

#	Feature	User Story
2	Generate Reports	As an administrator, I want to generate reports so that institutional performance can be monitored.

These stories matter because faculty and administrators are the ones who keep academic data accurate and current — their stories directly drive the system's data-entry and reporting modules, while student stories drive the self-service and information-access modules.

3. Software Development Process Model

3.1 Selected Model: Agile (Scrum)

The KL ERP System is developed using the Agile methodology, specifically the Scrum framework. Agile is an iterative approach in which software is built in small increments called Sprints, with each sprint focused on incorporating customer feedback, continuous improvement, and rapid, working delivery rather than a single large release at the end of the project.

3.2 Agile Development Phases

- Requirement Gathering — collecting needs from students, faculty, and administrators.
- Sprint Planning — breaking the project into manageable, time-boxed increments.
- Design — architecting modules, database structures, and UML models.
- Development — implementing features within each sprint.
- Testing — verifying each increment before it is accepted.
- Review — collecting stakeholder feedback at the end of each sprint.
- Deployment — releasing working increments to users.
- Maintenance — fixing issues and refining features post-release.

3.3 Why Agile Fits the KL ERP System

Agile was chosen over a traditional plan-driven model such as Waterfall for three practical reasons specific to this project:

- Requirements change frequently — academic policies, examination rules, and fee structures are revised often, and an iterative model absorbs these changes without derailing the whole project.
- Multiple departments provide continuous feedback — admissions, examinations, and finance teams each have different priorities, and Agile's sprint reviews give them a regular channel to shape the system.
- Modules can be developed incrementally — Student, Faculty, Course Registration, Attendance, Examination, and Fee modules are relatively independent, making them well suited to parallel, incremental delivery with faster testing and earlier feedback on each feature.

4. Programming Language and Technologies

The KL ERP System is built on a conventional, well-supported enterprise web-application stack, chosen for reliability, scalability, and the strength of its surrounding developer ecosystem.

4.1 Selected Language: Java

Java is the primary implementation language for the system's backend logic. It is an object-oriented, platform-independent language widely used for enterprise and web-based applications, and it was selected for four reasons: it is platform independent, it is secure and reliable, it supports large-scale applications well, and it benefits from strong community and tooling support.

4.2 Technology Stack

Frontend	HTML, CSS, JavaScript, React.js
Backend	Java with Spring Boot
Database	MySQL
Version Control	GitHub

React.js provides a responsive, component-based frontend for students, faculty, and administrators. Spring Boot supplies a mature, convention-driven backend framework for building REST APIs and handling business logic in Java. MySQL offers straightforward relational storage for structured academic and financial records, with easy integration into the Spring Boot backend. GitHub is used for version control and collaborative development across the team.

5. Implementation Strategy

The implementation of the KL ERP System follows a clear, step-by-step plan that mirrors the Agile phases described earlier, moving from analysis through to long-term maintenance.

5.1 Requirement Analysis

Requirements are collected from students, faculty, and administrators, and consolidated into a Software Requirements Specification (SRS) that captures functional and non-functional expectations for the system.

5.2 System Design

The system architecture, UML diagrams, and database structure are designed at this stage, translating gathered requirements into a concrete technical blueprint (see Section 8 for the use case and class diagrams produced during this phase).

5.3 Development

The core modules are built incrementally: Student, Faculty, Course Registration, Attendance, Examination, Fee, and Timetable modules, each developed and integrated sprint by sprint.

5.4 Testing

- Unit Testing — verifying individual components in isolation.
- Integration Testing — verifying that modules work correctly together.
- System Testing — verifying the complete, assembled system.
- User Acceptance Testing — confirming the system meets real user needs before release.

5.5 Deployment

The ERP system is installed on production servers, databases are configured, and end users are trained on the new workflows before go-live.

5.6 Maintenance

After deployment, the team fixes defects, upgrades features, and continues improving performance and security in response to ongoing usage and feedback.

6. Evaluation Strategy

The evaluation strategy is a method used to assess whether the KL ERP System meets its functional and non-functional requirements. Rather than testing only whether features work, it also verifies how well the system performs, how secure it is, how usable it is, and how reliable it remains under real-world conditions. Evaluation is organized around five criteria:

Functional	Login validation, course registration, attendance, marks entry, and fee payment all behave correctly against specified requirements.
Performance	Response time, throughput, and scalability are measured under realistic and peak academic-term loads.
Security	Authentication, authorization, data encryption, and secure communication protect institutional and personal data.
Usability	Ease of use, interface quality, and accessibility are assessed with real students, faculty, and administrators.
Reliability	Availability, error handling, and backup-and-recovery procedures ensure the system stays dependable.

Together, these five criteria give a rounded picture of system quality: functional correctness confirms the system does what it should, while performance, security, usability, and reliability confirm it does so quickly, safely, comfortably, and consistently.

8. UML Diagrams

Two complementary UML diagrams are used to document the KL ERP System's design: a use case diagram, which captures the system's functional scope from the perspective of its three actors, and a class diagram, which captures the structural design of the system's core entities and their relationships.

8.1 Use Case Diagram

The use case diagram below shows the three actors — Student, Faculty, and Administrator — and the functionality each interacts with inside the KL ERP System boundary. Student use cases (blue) cover self-service features such as secure login, course registration, attendance viewing, examination results, and fee payment. Faculty use cases (green) cover attendance recording, course material uploads, and marks entry. Administrator use cases (orange) cover student record management and report generation.

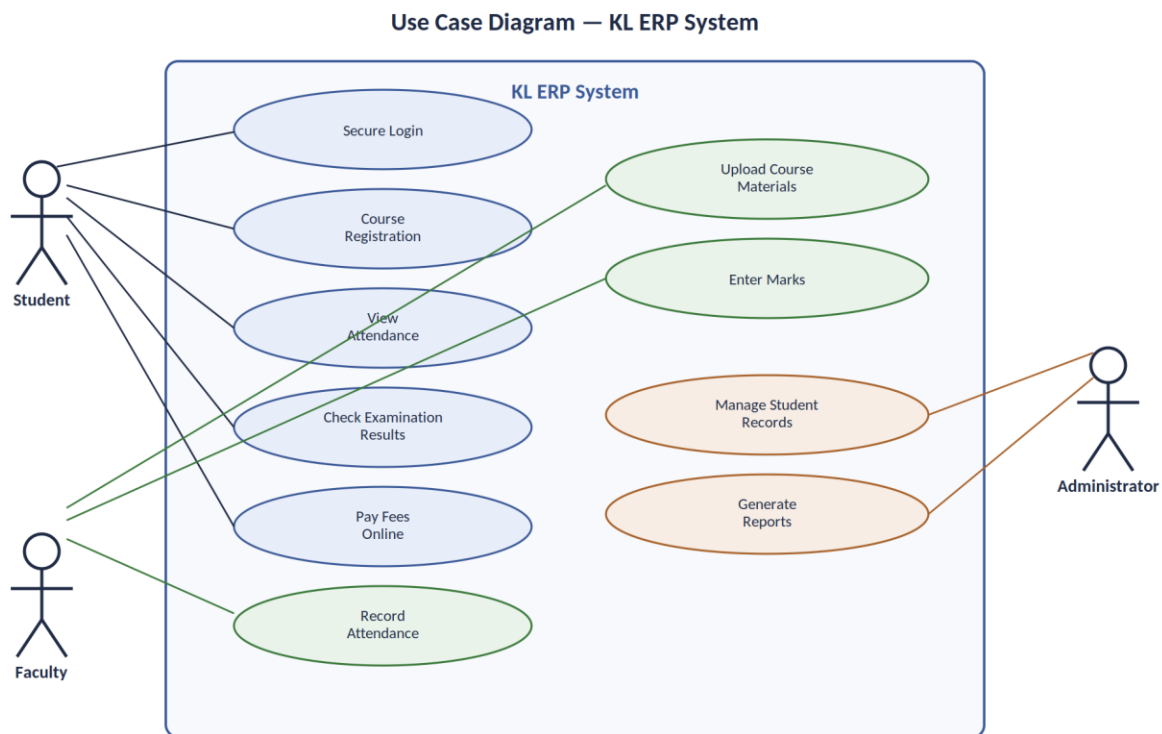


Figure 1: Use Case Diagram — KL ERP System

8.2 Class Diagram

The class diagram below models the core entities of the system. A base User class holds shared attributes and behaviour (login, logout), from which Student, Faculty, and Administrator inherit role-specific attributes and operations. Student associates with Course (registration), Attendance, Examination, and Fee; Faculty associates with Course (teaching), Attendance, and Examination; and Administrator associates with Report generation.

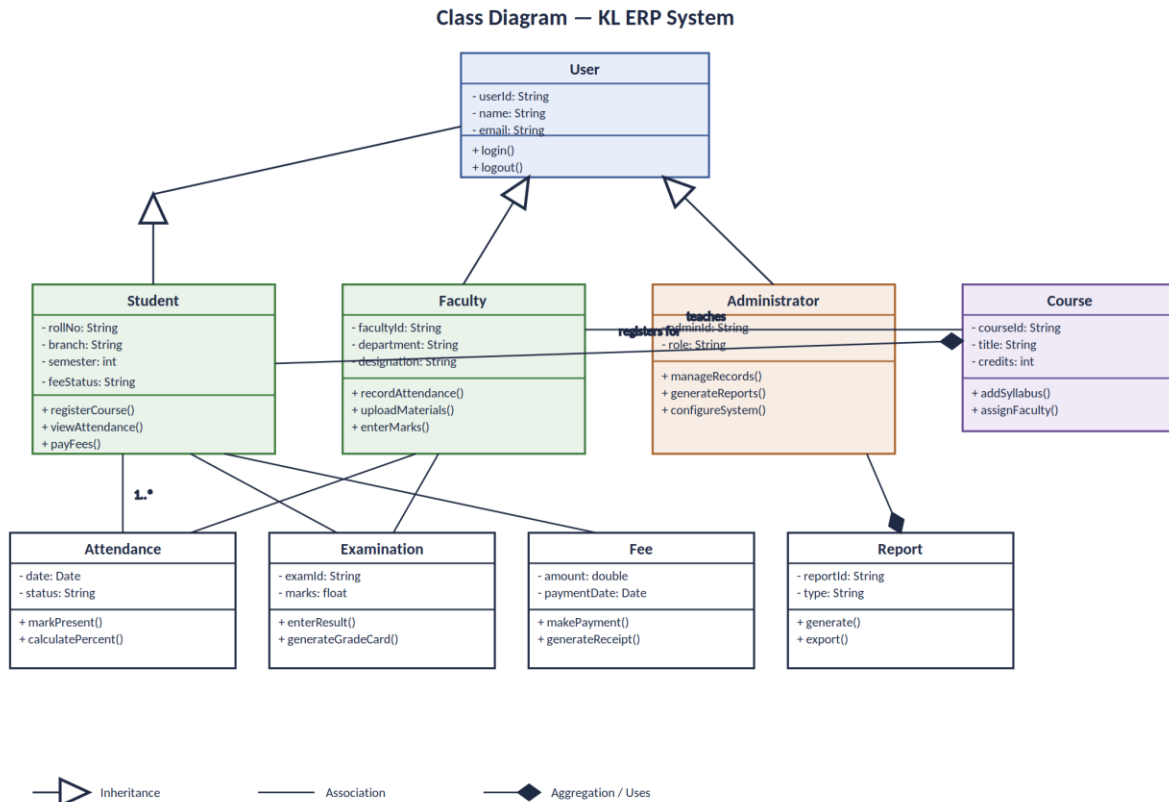


Figure 2: Class Diagram — KL ERP System

9. Conclusion

The KL ERP System is an integrated academic management solution designed to streamline institutional operations by bringing admissions, course registration, attendance, examinations, fee payment, and communication onto a single platform.

- User Stories — grounded the design in real requirements from students, faculty, and administrators.
- Agile Scrum — provided a flexible, iterative development process suited to changing academic requirements.
- Java Platform — offers a reliable, secure, and scalable implementation stack (Java, Spring Boot, MySQL, React.js).

- Structured Strategy — a clear implementation and evaluation strategy helps ensure successful development and deployment.

Taken together, these elements are expected to deliver centralized management, improved communication between stakeholders, reduced paperwork, faster processing of academic activities, and a system that is secure and scalable enough to grow with the institution.